

Q2 代码架构与运行逻辑完全指南

VLSI 布图规划设计 — 固定正方形轮廓下的 HPWL 最小化

2026 年第七届”华数杯”大学生数学建模竞赛 B 题

2026/08

目录

1 项目总体架构	2
1.1 目录结构	2
1.2 执行顺序	3
1.3 核心设计理念	3
2 Q2 求解流程图	4
2.1 流程图说明	5
3 Phase 1: C++ 核心 floor_plan.hpp (Q2 视角)	5
3.1 Q2 相关的关键结构	5
4 Phase 2: 两阶段退火引擎 sa.hpp	5
4.1 run(): 可行性阶段	5
4.2 run2(): 精修阶段	6
5 Phase 3: CLI main.cpp	6
6 Phase 4: 多起点并行求解 q2_solver.py	6
7 测试套件 test_main.cpp (1135 断言)	7
8 关键设计决策	7
8.1 为什么多起点并行取最优?	7
8.2 为什么 --seed 显式化?	7
8.3 为什么 run() 需要 reset 上限?	7
8.4 已知弱点 (如实记录, 当前为冻结基线)	7

9 实际运行结果	8
9.1 指标汇总（多起点 8 轮并行取优， $\alpha = 0.5$ ，dead_ratio=0.15）	8
9.2 多轮统计（8 轮 HPWL 分布，评估稳定性）	8
10 文件依赖关系	8
11 运行指南	8
11.1 环境要求	8
11.2 完整运行流程	9
11.3 常见问题	9
12 合规清单	9

1 项目总体架构

1.1 目录结构

```
1 The_7th_Huashu_Cup.../
2 |-- data/raw/                # 原始附件副本 n100/n200/n300 (只
   读)
3 |-- cpp_solver/              # C++ 共享核心 (Q1/Q2/Q3 共用, 单套
   测试)
4 |   `-- src/
5 |       |-- floor_plan.hpp    # B*-Tree 拓扑 + Skyline 解码 + 解析
   器
6 |       |-- sa.hpp            # 快速模拟退火引擎 (两阶段 + 收敛
   log + 快照)
7 |       |-- main.cpp          # CLI (mode/alpha/dead_ratio/--log
   /--seed)
8 |       |-- test_main.cpp     # 白盒测试 (1135 断言)
9 |       `-- test_oracle.hpp   # 独立参考解码器 / 合法性校验 / 暴力
   枚举
10 |-- common/
11 |   |-- visualize.py          # 共享绘图基础 (.rpt/.log/snap 解
   析)
12 |   `-- verify.py            # 独立合法性复核 (无重叠/尺寸/越界)
13 |-- q2/                      # 问题 2 客制层 (<-- 本文件)
14 |   |-- q2_solver.py         # 多起点并行求解: 调 bin/main, 取
   HPWL 最优
15 |   |-- output/
16 |   |   |-- q2_metrics.csv    # 指标汇总 (side/HPWL/利用率/合法性/
   seed)
17 |   |   |-- q2_n100.rpt       # 最优布局坐标
18 |   |   `-- q2_n100.log      # 收敛轨迹 + 9 等分过程快照
19 |   `-- visualization/
20 |       |-- plot_floorplan.py # 最终布图 (轮廓框 + HPWL 标注)
21 |       |-- plot_convergence.py # 收敛曲线 (run/run2 双阶段)
22 |       |-- plot_snapshots.py # 9 等分过程快照
23 |       `-- figs/<时间戳>/<chip>/ # 每芯片 11 张图 (保留全部历史)
24 |-- q1/ q3/ q4/              # 其余问题 (共享核心 + 各自客制层)
25 `-- paper/                   # 论文写作
```

1.2 执行顺序

执行顺序

1. `cd cpp_solver && make` (编译 `bin/main`, 0 警告)
2. `make test` (可选, 白盒测试回归, 1135 断言)
3. `conda activate math`
4. `python q2/q2_solver.py` (多起点并行求解 + 汇总 + 出图)

1.3 核心理念

多起点独立退火 (Multi-start SA): Q2的 HPWL 目标在紧轮廓 (死区 15%) 下搜索地形崎岖, 单次退火结果依赖种子。采取 N 个独立 SA 实例 (不同随机种子) 并行搜索、取 HPWL 最优——这是优化领域标准做法, 且天然可并行 (进程级, M5 10 核吃满)。

两阶段退火: `run()` (自适应 α + 轮廓压力找可行解) $\rightarrow$ `run2()` (低温精修 HPWL)。固定轮廓下先确保合法性、再优化线长。

可复现性: 所有轮次以 `--seed` 显式注入随机种子 (`seed_base` 记录于 CSV), 结果可精确复现。

2 Q2 求解流程图

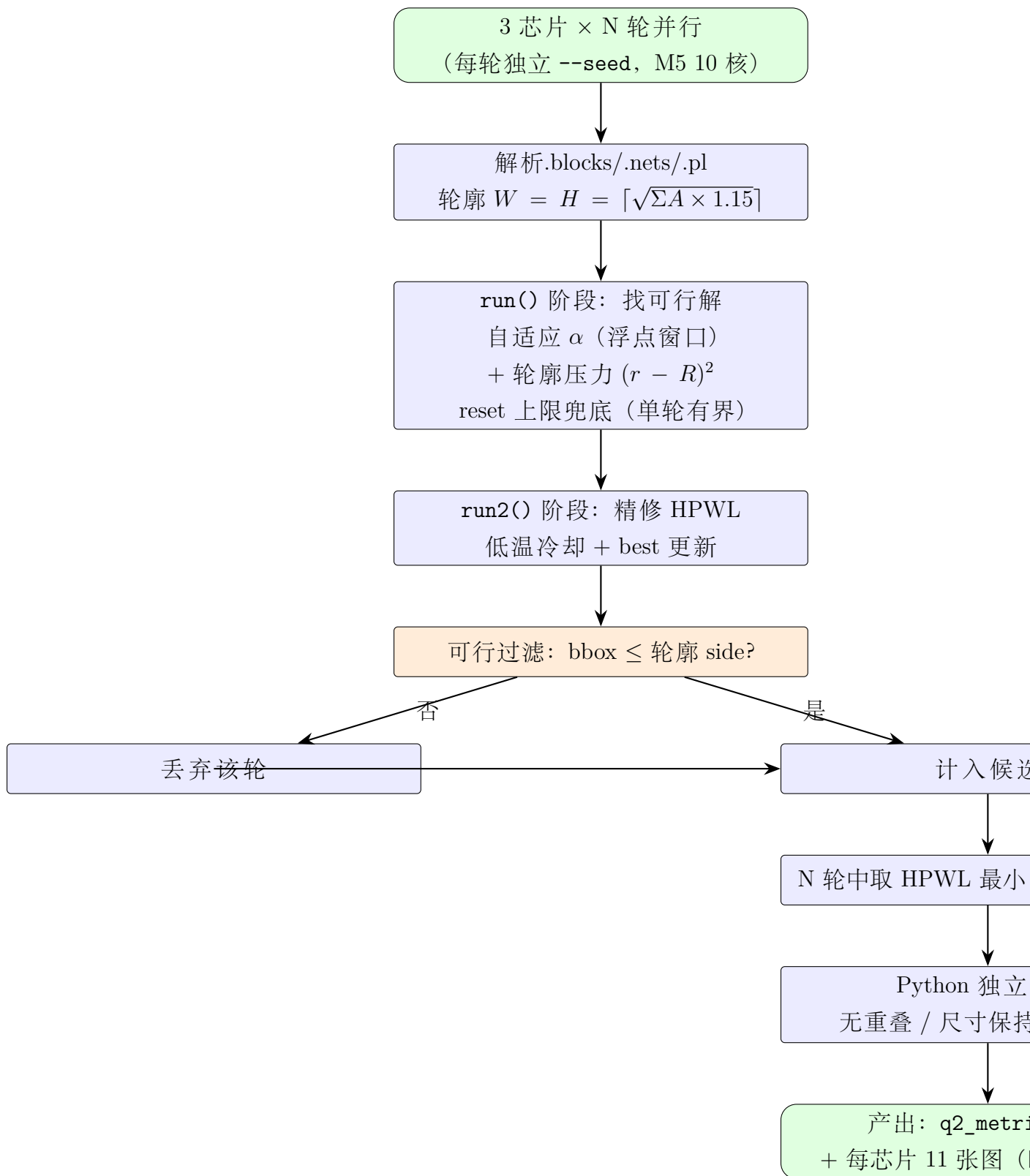


图 1: Q2 多起点并行求解流程图

2.1 流程图说明

1. **多起点并行**: 每个 `bin/main q2 ... --seed s` 是一个独立退火实例（不同初始树与扰动序列），三芯片并行、每芯片内部 N 轮并行（峰值 9 进程）。
2. **run() 阶段**: 以面积与长宽比加权代价驱动，自适应 α 随可行性窗口滑动（浮点连续化），把布局压进轮廓；reset 机制保证“找不到可行解”时单轮时间有界（上限 $N/16 + 1$ 次温度重置）。
3. **run2() 阶段**: 从 run 的 best 出发，低温精修 HPWL。
4. **可行过滤**: bbox超出轮廓的轮次直接丢弃（不污染最优）。
5. **独立复核**: Python侧重新解析文件、逐对检查重叠/尺寸/越界，不信任 C++ 单侧结果。

3 Phase 1: C++ 核心 floor_plan.hpp (Q2 视角)

3.1 Q2 相关的关键结构

- `cost()`: 返回 (HPWL, W, H); HPWL 按引脚几何中心 (块: $2x + w$, 终端: $2x$), 全量扫描线网 (无增量近似)。
- `feas(cost)`: Q2 模式检查 $W \leq _W \ \&\& \ H \leq _H$ 。
- 解析器: 完整读取 `.blocks/.nets/.pl` (HPWL依赖终端预计算 `NET::update`, 线网按 id 排序)。
- **NODE 位压缩**: 10-bit 父子/左右指针 + 旋转位 (无符号存储, 消除有符号位移疑虑)。

4 Phase 2: 两阶段退火引擎 sa.hpp

4.1 run(): 可行性阶段

- 代价 $\text{norm_cost} = \alpha \frac{A}{\bar{A}} + \beta \frac{\text{HPWL}}{\bar{H}} + (1 - \alpha - \beta) \frac{(r - R)^2}{\bar{r}}$ 。
- $\alpha = \alpha_{\text{base}} + (1 - \alpha_{\text{base}}) \frac{N_{\text{feas}}}{N}$ (浮点连续化, 随可行性窗口滑动); β 在可行后递增施加轮廓压力。
- reset 机制: $\text{tot_feas} = 0$ 时温度重置, 上限 $N/16 + 1$ 次, 保证有界。
- 每温度层写收敛 log + 记录 best 树快照 (9 等分输出)。

4.2 run2(): 精修阶段

- 从 run 的 best 出发，代价 true_cost（面积与 HPWL 加权）。
- 初始温度 $_init_T/50$ ，低温快速下山；best 只在可行且更优时更新。
- tot_feas 初始化为当前 best 的可行性（修复：避免从可行解出发仍触发 reset 空转）。

5 Phase 3: CLI main.cpp

```
1 ./bin/main q2 <alpha> <blocks> <nets> <pl> <rpt> [dead_ratio]
2      [--log <file>] [--feas-only] [--seed <n>]
```

- <dead_ratio>: 轮廓死区比例（Q2 固定 0.15；Q3 二分传不同值）。
- --feas-only: 只跑 run() 判定可行性（Q3 专用，找到首可行即返回）。
- --seed: 随机种子（默认当前时间），配合 seed_base 复现。
- 输出.rpt: cost / hpwl / area / W H / time /每模块坐标。

6 Phase 4: 多起点并行求解 q2_solver.py

1. 任务分解：(chip, round, seed) 全部提交线程池（3 芯片 $\times$ 3 轮并行 = 峰值 9 进程，M5 10 核吃满）。
2. 种子分配：seed = seed_base + chip_idx*1000 + round（确定性、互异、可复现）。
3. 每轮独立输出 q2_<chip>_r<round>.rpt/log。
4. 可行过滤（bbox $\leq$ side）后取 HPWL 最小轮次为 best。
5. 汇总 q2_metrics.csv（含 seed_base 复现依据）。
6. 出图：figs/< 时间戳 >/<chip>/ 每芯片 11 张（9 快照 + 收敛 + 布图轮廓框）。

7 测试套件 test_main.cpp (1135 断言)

- Q2 专属用例: test_q2_run_snapshots(run 阶段 9 快照格式与坐标合法)、test_q2_feas_only (可行/不可行实例判定正确)、test_q2_end_to_end_hpwl (SA 结果 HPWL 与独立 oracle 全等)。
- 共享 oracle 体系: 独立 HPWL、暴力枚举、定义校验、穷举差分。
- make test (1135) + make test-san (ASan/UBSan) 双轨。

8 关键设计决策

8.1 为什么多起点并行取最优?

Q2 轮廓紧 (死区 15%)，单次退火结果依赖种子: n300 单轮 HPWL 实测波动 574k~3152k。N 个独立实例并行、取 HPWL 最优——搜索质量单调不差于单实例，且进程级并行吃满 10 核 (实测 96-99%)。

8.2 为什么 --seed 显式化?

默认 srand(time) 使结果不可复现 (论文评审硬伤)。显式种子 + seed_base 记录后，任意结果可精确复现。

8.3 为什么 run() 需要 reset 上限?

差种子时 run() 可能长时间找不到可行解 (温度重置循环)。上限 $N/16 + 1$ 次重置保证单轮有界 (避免“卡死”)，配合多起点对冲概率。

8.4 已知弱点 (如实记录，当前为冻结基线)

- run() 可行性搜索对 n300 不稳定 (好/差种子单轮 1.5~7min 波动)。
- 8 轮取优的轮间方差大: n100 spread 192%、n300 spread 449%; 中位数远差于最优 (30~46%) ——best 存在运气成分。
- 提升方向 (不改变核心算法): 增加轮数、run2 精修强度调优、分层筛选种子——留待参数优化阶段。

9 实际运行结果

9.1 指标汇总（多起点 8 轮并行取优， $\alpha = 0.5$ ，dead_ratio=0.15）

芯片	轮廓 side	HPWL	面积利用率	理论上限	合法性
n100	455	235183	0.867	0.8696	合法
n200	450	403295	0.868	0.8696	合法
n300	561	574322	0.868	0.8696	合法

面积利用率 = 模块总面积 / side²，理论上限 = $1/1.15 = 0.8696$ （死区 15%）——实测逼近理论极限，布局紧凑度最优。

9.2 多轮统计（8 轮 HPWL 分布，评估稳定性）

芯片	min	中位数	max	spread	中位/最优
n100	235183	304632	687833	192%	1.30
n200	403295	560428	585763	45%	1.39
n300	574322	838620	3152712	449%	1.46

10 文件依赖关系

```
1 q2_solver.py
2 |-- cpp_solver/bin/main          (编译自 main.cpp + sa.hpp +
   floor_plan.hpp)
3 |-- data/raw/{n100,n200,n300}.{blocks,nets,pl}
4 |-- common/{visualize,verify}.py
5 |-- q2/visualization/plot_{floorplan,convergence,snapshots}.py
```

11 运行指南

11.1 环境要求

- g++ (Apple clang, -std=c++17, 无第三方依赖)。
- conda 环境 math (Python 3.9+, matplotlib)。
- TeX Live 2026 (xelatex + ctex, 编译本指南)。

11.2 完整运行流程

```
1 cd cpp_solver && make && make test      # 编译 + 白盒测试
2 cd ..
3 conda activate math
4 python q2/q2_solver.py --repeats 8      # 多起点并行求解 + 汇总 + 出
   图
5 python q2/q2_solver.py --skip-solve     # 只重出图/汇总（复用已有结
   果）
```

11.3 常见问题

1. 图没更新：求解只写 output/，画图每次新建时间戳目录——看最新目录。
2. HPWL 想更好：增大 --repeats（纯参数，不改变算法）。
3. 复现：CSV中 seed_base + 每轮 --seed 公式（seed_base + chip_idx*1000 + round）即可复现。
4. CPU 利用：峰值 9 进程（3 芯片 × 3 轮），M5 10 核吃满。

12 合规清单

- 原始数据 data/raw/ 为只读副本，求解器不修改输入文件。
- 全部结果可通过 make test（1135 断言）+ make test-san 复现。
- 随机种子显式管理(--seed + seed_base), 结果可复现; 论文数值以 q2_metrics.csv 与 figs/< 时间戳 >/ 为准。
- 已知弱点（轮间方差）如实记录，未作任何美化。